The End of Alignment

Technical Field Guide

From Milestone to Production

Adrian McPhee

Versioned companion

v0.12.5

About This Companion

This is the implementation companion to *The End of Alignment*. The main book establishes the organisational argument and contains the complete policy-renewal packet. This guide preserves the detailed protocol for temporary delivery authority, concurrent human and agent work, exact identity, production evidence, recovery, and retirement, then annotates how that protocol applies to the packet without reproducing it.

The companion is versioned because the engineering practice will continue to change. Its names are replaceable. The required outcomes are not: authority remains explicit, shared meaning is protected, production claims bind to exact evidence, and temporary state retires.

Contents

Technical Field Guide: From Milestone to Production

This companion is for the people who must carry a governed change into production. The Field Guide states the protocol and closes with annotations that apply it to the main book's complete worked packet. It is written so an implementer can begin here even if the book arrived in another inbox. Directors can use it to test a delivery claim beyond the board pack; CTOs, unit leads, engineers, and orchestrators can inspect the machinery in full.

The problem this protocol solves

Software and agents make it cheap to start several changes at once. They do not make it cheap to decide who may alter shared meaning, prove which exact result was reviewed, integrate independent work, or recover when a writer stops halfway through. Without an explicit protocol, concurrency creates branches, environments, flags, schemas, and claims whose ownership must later be reconstructed through conversation.

A *lane* carries one independently useful outcome through that system. It has one writing owner and a defined retirement condition. A *lease* grants that lane bounded permission to mutate an exact physical or semantic resource. The distinction matters: access to a repository or environment establishes capability, not authority. Lanes make the outcome and temporary owner visible; leases make the authorised surface visible.

Names form the control plane because every later proof depends on knowing which object is being discussed. “The renewal change” cannot tell an orchestrator which promise, slice, source candidate, artefact, flag state, audience, or evidence run somebody means. Stable human names joined to immutable machine identities allow the system to route authority and allow a reviewer to reproduce the claim.

The protocol also refuses the word “delivered” as a collapsed state:

> Integrated source ≠ deployed artefact ≠ customer exposure ≠ observed outcome.

Source can be present on trunk while no production artefact serves it. The artefact can be deployed while exposure remains disabled. Exposure can begin while the promised outcome remains unproved. Each transition has a named authority, an exact identity, evidence, and a reversal or retirement rule.

The main book gives the business reason for durable unit ownership and defines the executive chain from portfolio intent to outcome evidence. This guide begins where ordinary concurrency becomes difficult and turns that chain into an executable sequence: names, lanes, leases, joined proof, exact identities, recovery, and retirement.

Names are the control plane

Concurrency fails when different actors use the same word for different things or different words for the same thing. "The payments change," "the new release," and "the agent branch" are not identities. They force an orchestrator to reconstruct meaning from conversation, which is the problem the protocol exists to remove.

Every durable and temporary object receives a stable, outcome-shaped name plus an exact machine identity where one exists. The name tells a person what the object is for. The identity lets the system prove which object it means.

Durable names cover the product, customer promise, process, unit, contract, deployable, and feature-flag namespace. Temporary names cover the milestone, slice, lane, lease, worktree, changeset, candidate, deployment operation, evidence run, and recovery item. A useful lineage might read:

> Product: merchant refunds
>
> Milestone: confirmed refund within three days
>
> Slice: restart-safe provider retry
>
> Lane: restart-safe-provider-retry, attempt two
>
> Lease: exact payment-adapter paths, test environment, and integration interval
>
> Candidate: one immutable source commit and build digest
>
> Exposure: release flag for the named customer cohort
>
> Evidence: production verification against that candidate and cohort

Lane names describe the result, never the worker, model, technology, or generic activity. A lane called “agent four backend” says nothing about the product and becomes meaningless when ownership transfers. A lane called “restart-safe-provider-retry” preserves its purpose through a human-to-agent or agent-to-agent handoff.

A lease name includes the exact resource and holder. A path prefix alone may be insufficient: two lanes in different folders can still change the same public contract, generated schema, rollout flag, or production state. The lease registry therefore records physical and semantic conflicts. A worktree name identifies isolated mutable state. A candidate identity identifies immutable review state. A deployment identity identifies what can run. An exposure identity identifies who can experience it. Evidence binds to all three.

The orchestrator joins these names into a traceable graph. Given a customer-visible behaviour, it can find the product promise, process owner, milestone, slice, lane, source candidate, serving artefact, flag state, and proof. Given a stale worktree or active credential, it can find the lease, holder, purpose, and retirement condition. Names make the system addressable. Leases make action authorised.

Design lanes around outcomes

A lane is not "backend work," "agent three," or "phase two." It has the name of the user or operator result it is meant to close: recoverable signed update, restart-safe report, completed refund confirmation. The lane record names the starting source identity, exact writable paths, read-only dependencies, forbidden adjacent scope, required leases, acceptance criteria, seam proof, integration route, decision fences, and retirement condition.

One lane has one writing owner. Specialists can inspect a frozen candidate from security, domain, compliance, or performance perspectives, but they return evidence to the owner. If two people or agents mutate the same candidate concurrently, responsibility for the resulting state is already ambiguous.

Parallel writing is justified only by independent outcomes. The paths must be disjoint, the semantic contracts compatible, the workspaces and build directories isolated, the gates independent, the integration order explicit, and any scarce machine or environment capacity separately leased. Available agent capacity is not a reason to create work.

These controls let five, ten, or fifty teams work in one monorepo without standing on one another's toes. They do not avoid conflict by pretending the repository is partitioned into private kingdoms. They make durable ownership legible, admit only bounded slices, and serialise the few resources that are genuinely shared.

Leases cover meaning as well as files

The simplest lease grants mutation authority over an exact source path. File overlap is only one form of conflict. Two lanes can edit disjoint files while changing the same public schema, generated model, feature-flag namespace, production environment, signing channel, or customer promise.

A practical system leases whatever can invalidate another lane's work or evidence. That includes paths, workspaces, changesets, integration intervals, schemas, generators, build capacity, performance hosts, deployment environments, credentials, evidence records, and external communication authority.

Each lease has an identity, holder, mode, permitted operation, lifetime, starting condition, conflicts, and release rule. Shared means explicitly designed for concurrency. It never means nobody decided.

This does not require a central lease office. Units and resource owners issue most leases locally; explicit conflict rules make routine grants mechanical. Work inside a coherent unit boundary should proceed without a meeting. Human judgement remains for real authority fences: cross-unit promises, public contracts, risk decisions, contested shared resources, and choices the orchestrator has no right to make. A committee for routine leases merely rebuilds approval bureaucracy inside delivery.

Leases also make recovery possible. If an agent stops halfway through a change, the orchestrator can identify the exact workspace, branch, files, environment state, and evidence under its custody. It can transfer the lane through a recorded handoff or reject the candidate without guessing what else the agent touched. Temporary authority ends mechanically when the result is integrated or abandoned. Branches, worktrees, flags, migration versions, and recovery artefacts do not become a graveyard of forgotten intentions.

Start at the seam

Horizontal task decomposition is attractive to agents. One can build the interface, another the service, another the schema, another the tests. Each produces output. None owns the joined behaviour. The defects that matter appear between them: wrong identity, stale state, missing transaction boundaries, retries that duplicate an action, feature flags that expose only half the path, or a deployment that serves a different candidate from the one tested.

A slice begins with the real initiating gesture and the intended production result. The owner maps every risk-bearing boundary between them and creates one assertion that can fail when the connection is broken. A customer clicking “refund” should reach the domain decision, payment instruction, idempotent provider interaction, reconciliation, durable state, and customer confirmation. Component tests remain useful. They cannot substitute for the joined seam.

The implementation then changes the smallest complete path, regenerates declared consumers, runs focused and contract checks, records the exact candidate, reconciles it with current trunk, reruns affected tests, integrates normally, and verifies the resulting remote identity. Product truth moves only as far as the evidence supports.

The sequence is mundane. It is also the difference between an agent producing plausible source code and the company changing safely.

Trunk is integration truth

Trunk-based development keeps one current source truth. Short-lived branches and worktrees isolate execution, then integrate complete slices frequently. A rejected integration because another lane landed first is normal: the later lane reconciles against the new trunk, regenerates affected state, reruns its checks, and tries again. Any semantic or identity change creates a fresh candidate, so reviewers inspect the reconciled result rather than the obsolete one. The lane does not force its version of reality onto everyone else.

Trunk-based development does not mean unfinished behaviour is visible to customers. It separates integration from deployment and exposure:

integrated source $\neq$ deployed artefact $\neq$ customer exposure

Long-lived product branches create parallel companies. The longer they live, the more each accumulates assumptions about contracts, schemas, dependencies, and behaviour that have not met the rest of the system. A release train postpones the meeting and then calls the collision an integration phase.

Frequent integration makes boundary mistakes visible while they are still small. It also lets independent units move at independent rates. A monorepo can contain many deployables, and one commit can produce several of them. Repository topology, ownership topology, module topology, deployment topology, and exposure topology are separate choices. A monorepo decides only where source is integrated.

Feature flags separate delivery from exposure

A feature flag allows code to reach trunk and production before a customer receives the new behaviour. That is powerful only when the flag has an owner and a semantic purpose.

A release flag is temporary and disappears after stable exposure. An experiment flag records the hypothesis and the date on which evidence will decide it. An entitlement flag expresses durable commercial policy and belongs to the relevant domain. An operational safety flag records who may act during an incident and preserves the audit trail. Every flag names the guarded outcome, default behaviour, targeting authority, metrics, rollback, creation date, and removal condition. Both the enabled and disabled paths are tested while both exist.

Flags cannot hide incompatible schemas or incomplete contract migrations. They are an exposure mechanism, not an alternative to architecture. A flag left without an owner becomes a permanent fork in the running company, exactly the divergence trunk-based development was meant to prevent.

Contracts make cross-team movement ordinary

When one unit changes a public interaction, the preferred sequence is compatible expansion and contraction. The provider adds a compatible version and deploys it. Consumers migrate independently. Observation proves the old version has no remaining use. The provider removes it in a final slice. Each step is green, reversible, and owned.

An atomic change across provider and consumer can occasionally be safer, especially inside a monorepo. In that case one lane receives explicit leases from both owners and each owner reviews its boundary. The exception is visible. It does not become an argument that ownership never mattered.

Reusable capability should remain with its origin until a second real consumer needs a stable contract. Extracting every repeated helper into a central platform creates a queue disguised as reuse. Platforms earn their place by providing services units voluntarily consume under explicit promises. They do not acquire governance authority merely because many teams depend on them.

The complete execution state machine

The protocol can now be stated as a state machine rather than a collection of practices. The normal path is deliberately strict about identity and deliberately flexible about implementation.

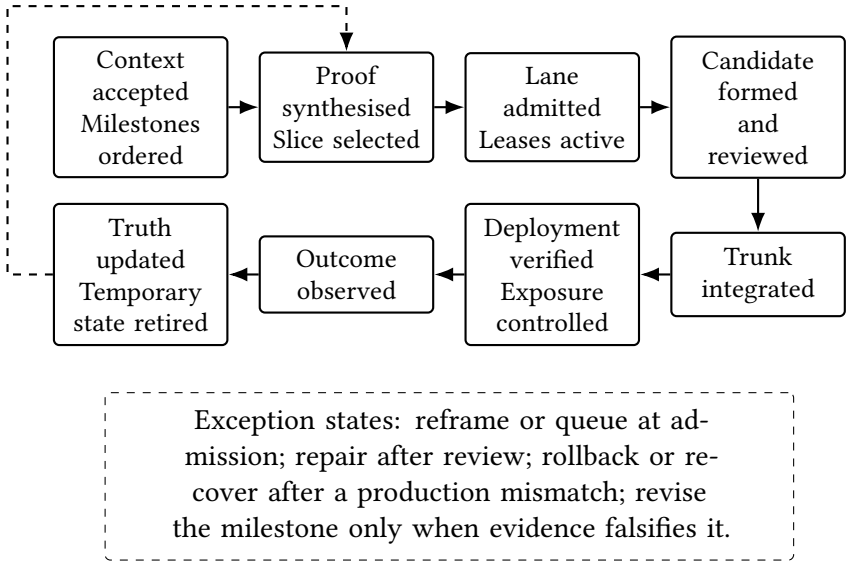

Figure FG.1: The complete execution state machine.

1. **Context accepted.** A product thesis and promise sit inside the corporate strategic context. The current milestone traces either to a funded bet and its strategy and goal, or to an explicit operating obligation. The product authority records the customer, promise, product vision, quality bar, boundaries, and non-goals. Implementation is not yet authorised merely because this context exists.
2. **Milestones ordered.** The unit roadmap expresses a dependency-aware sequence of complete outcomes. One milestone becomes current when upstream truth and work-in-progress guards allow it.
3. **Proof synthesised.** The unit compares the stable milestone with current product, process, source, deployment, and runtime truth. Open gaps and exact evidence are visible.

4. **Slice selected.** The process owner chooses one independently useful movement with an initiating gesture, complete result, exclusions, seam proof, and acceptance criteria.
5. **Lane admitted.** The orchestrator confirms product authority, scope, one writing owner, a healthy starting identity, a viable integration route, and a mechanical retirement condition.
6. **Leases active.** Exact path, workspace, changeset, contract, environment, credential, and other required resource leases are granted. Mutation may begin only now.
7. **Candidate formed.** The writer implements the joined path, changes canonical source, regenerates consumers, and passes the declared focused, seam, contract, and risk-shaped gates. One immutable candidate identity is declared.
8. **Candidate reviewed.** Required reviewers inspect that exact identity. A returned candidate goes back to the same lane for repair. A rejected approach receives an explicit disposition rather than lingering as an alternative branch.
9. **Trunk integrated.** The candidate reconciles with current trunk. If another lane landed first, it rebases or otherwise reconciles semantically, regenerates affected state, and reruns the relevant checks. If identity or meaning changes, the result becomes a new candidate and repeats the required review. One brief integration lease serialises the final mutation of trunk.
10. **Deployment verified.** The built artefact is bound to the integrated source identity. Deployment records the environment and immutable artefact. A successful pipeline is not production proof until the serving identity is confirmed.
11. **Exposure controlled.** The owning authority changes the feature flag, experiment, entitlement, or operational-safety state for an exact audience. Integrated, deployed, and exposed remain separate facts.
12. **Outcome observed.** Production behaviour is checked against the milestone proof. The unit records what became true, what drift re-

mains, and whether the result changes the product, process, contract, risk, or strategic premise.

13. **Truth updated and temporary state retired.** Product and process truth advance only as far as evidence. Leases are released. The worktree and branch disappear. Temporary flags and migration versions receive an expiry or are removed. The next slice is selected from a fresh reconciliation.

The exception paths matter as much as the happy path. A failed admission is reframed or queued. A lease conflict is serialised or redesigned. A product, legal, risk, cost, or public-promise choice is routed to its named authority. A failed review returns to the writing owner. A production mismatch triggers rollback or recovery against exact identities. Evidence that falsifies the milestone opens an authorised revision; it does not permit a team or agent to weaken the target silently.

Recovery requires a defined state. When workspace, lease, candidate, or production identity becomes ambiguous, mutation stops. Current state is inventoried and preserved. Useful work is restored to one owner or rejected recoverably. Only then are temporary resources released. No orchestrator should solve uncertainty by deleting the evidence of it.

The global invariants are short enough for a CTO to remember. Every current slice traces to an active bet or explicit operating obligation, an accepted product promise or process duty, and a milestone; every writing actor belongs to one lane; every mutation has an active lease; and exclusive leases do not overlap physically or semantically. Candidate identity remains fixed through review, production claims bind to exact serving and exposure identity, product truth never outruns proof, and a lane is not complete until its temporary inventory has retired.

Why this scales better than separation

Companies often split repositories, teams, and runtime services to reduce collisions. That can be useful for legal, operational, or scaling reasons. It does not resolve a process whose state and decisions still cross all the

boundaries. The coupling simply becomes harder to see, and the company pays for network failure, fleet operations, observability, versioning, and more specialised staff on top of the original coordination.

A monorepo can hold a modular monolith whose units build and release independently when ownership, state, contracts, deployables, and exposure are explicit. Conversely, a microservice estate remains distributed only in topology if every meaningful change requires several teams and a bridge call. The test of modularity is whether one part can change without negotiating the whole, not how many services exist.

The monorepo is the hardest and most useful case because it makes shared truth and accidental coupling visible. Teams integrate into the same source reality. Policy can check ownership and dependency manifests against actual imports, builds, deployables, and contracts. Atomic changes are possible where necessary. Independent release remains possible where boundaries are real.

The system scales by serialising contention while independent unit work proceeds concurrently. Contract transitions, integration intervals, shared generators, environments, and credentials are serialised because they are genuinely scarce or coupled. The orchestrator keeps that distinction current.

The delivery logic reduces to one path:

> Strategic context constrains product promises. Portfolio authority funds a bet or records an operating obligation; product and process authorities order milestones. Current evidence reveals the next outcome gap. The unit selects a vertical slice, the orchestrator admits its lane, and exact leases grant temporary authority. One writer forms an immutable candidate in an isolated worktree and reconciles it with trunk. Deployment binds artefact to source; flags control exposure. Production evidence updates truth, then leases and temporary state retire.

Without this last mile, the operating logic remains a description above the machinery through which the company changes.

The main book's worked policy-renewal packet instantiates the same logic at the process boundary: one definition, one published contract, one charter, and one report that traces declared meaning to production evidence. The closing annotations carry one of its findings through this protocol.

Applying the Protocol to the Worked Packet

The main book contains one complete policy-renewal packet: a process definition, a published contract, a unit charter, and a sample drift report. This companion does not reproduce those pages. It shows how an implementer carries their identities and decisions through the delivery protocol.

The four objects perform different jobs.

Packet object	Use in the delivery protocol
Process definition	Supplies the owned states, transitions, invariants, failure behaviour, and exact meaning that a milestone or repair must preserve
Published contract	Names the semantic seam, consumers, compatibility rules, failure behaviour, migration duties, and contract leases a change may require
Unit charter	Establishes durable process authority, reserved independent rights, exposure thresholds, escalation, and the limits inside which temporary delivery authority may be granted
Drift report	Joins declared meaning to source, deployable, configuration, audience, observation, human disposition, and the next authorised change

Carry finding F-041 into production

The packet's first drift finding says that amendment scores from `0.65` should enter referral, while the running rule refers only at `0.72`. The report identifies the affected cohort and gives the finding a human dispo-

sition: the implementation is wrong relative to the approved definition. That disposition creates a bounded operating obligation. It does not yet authorise somebody with repository access to change production.

A unit applying this guide would record the movement as follows.

1. **Name the outcome.** The milestone is not "change threshold." It is that every eligible amendment at or above `0.65` enters the declared referral path, with the affected cohort preserved and no unsupported amendment exposed.
2. **Admit one lane.** A lane such as `referral-threshold-repair` receives one writing owner, the finding and decision identities, acceptance criteria, and a retirement condition.
3. **Lease the real change surface.** The lane may need the rule implementation, configuration revision, contract tests, generated consumers, amendment-exposure flag, test environment, and a short integration interval. Disjoint files do not remove a conflict over the same rule or cohort.
4. **Form one candidate.** The source and configuration correction, back-test against the 218-cycle cohort, focused tests, joined seam proof, and rollback evidence bind to one immutable candidate. Any reconciliation that changes meaning creates a new candidate.
5. **Integrate, deploy, and expose separately.** Trunk identity, deployable digest, production configuration, and the stopped or staged cohort remain distinct. A successful pipeline does not close F-041.
6. **Observe the declared result.** Production evidence shows the applied threshold, referral transitions, affected audience, control operation, and any customer or operational consequence over the stated window. The unit lead then closes, repairs, rolls back, or revises the governing definition through a new authority decision.
7. **Retire temporary state.** The lane, leases, repair flag, test cohort, branch or worktree, and exceptional access disappear or receive a named continuing owner. The drift report records the exact evidence that closed the finding.

The same path applies to the packet's delayed terminal events and legacy cycles with missing source identity. The shape of the correction changes; the control logic does not. Meaning comes from the process and contract, permission comes from the charter and temporary grant, production truth comes from exact serving and audience identities, and closure comes from observed evidence plus retirement.

Do not copy the packet's identifiers into another process. Replace each with a real identity from the local process, source, deployment, configuration, and telemetry. A sample hash that survives into production documentation has achieved the opposite of provenance.

Use and Attribution

Copyright © 2026 Adrian McPhee. Published by Simple is Advanced. The companion may be read alongside the versioned book at simpleisadvanced.com/the-end-of-alignment/.